POO - Templates

Templates

Templates - concept

Template-urile sunt instrumente ce sunt regăsite în C++. Ele se folosesc de conceptul de a trimite ca și parametru un tip de date.

Astfel putem refolosi o metodă pentru mai multe tipuri de date, folosind același cod. De exemplu dacă avem nevoie de mai multe metode de sort care să aibă tipuri de date diferite, în loc să scriem codul de mai multe ori folosim un template.

Astfel se introduc 2 termeni noi:

- template
- typename (care poate fi schimbat cu class)

Logica de templates se realizează la compile time

```cpp
#include <iostream>
using namespace std;

template <typename X>
X exMax(X x, X y)
{
   return (x > y)? x: y;
}

int main()
{
  cout << exMax<int>(10, 5) << endl;  //int
  cout << exMax<double>(2.54, 9.21) << endl; //double
  cout << exMax<char>('o', 'r') << endl;   //char

  return 0;
}
```

Exemplu template max function

Exemplu sort

```cpp
#include <iostream>
using namespace std;

template <class T>
void bubbleSort(T a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = n - 1; i < j; j--)
            if (a[j] < a[j - 1])
              swap(a[j], a[j - 1]);
}

int main() {
    int a[5] = {10, 50, 30, 40, 20};
    int n = sizeof(a) / sizeof(a[0]);

    bubbleSort<int>(a, 5);

    cout << " Sorted array : ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;

    char b[5] = {'b', 'x', 'd', 'm', 'o'};
    int m = sizeof(a) / sizeof(a[0]);

    bubbleSort<char>(b, 5);

    cout << " Sorted array : ";
    for (int i = 0; i < m; i++)
        cout << b[i] << " ";
    cout << endl;

  return 0;
}
```

Class Templates

La fel ca și template-urile care definesc funcții, putem să avem template-uri care definesc clase. Acestea sunt utile în momentul în care o clasă definește structuri de date care sunt independente de tipul de date.

```cpp
#include <iostream>
using namespace std;

template <class T>
class myPair {
    T a, b;
  public:
    myPair (T first, T second)
      {a=first; b=second;}
    T getMax ();
};

template <class T>
T myPair<T>::getMax ()
{
  T retval;
  retval = a>b? a : b;
  return retval;
}

int main () {
  myPair <int> myObject1 (100, 75);
  cout << myObject1.getMax() << endl;
  myPair <char> myObject2 ('r', 'v');
  cout << myObject2.getMax() << endl;
  return 0;
}
```

Exemplu class

Template specialization

Având în vedere faptul că template-urile primesc ca și parametrii tipuri de date, putem să realizăm o logică diferită în template în funcție de tipul de date primit.

```cpp
#include <iostream>
using namespace std;

template <class T>
void example(T x)
{
   cout << "Main template example: "
        << x << endl;
}

template<>
void example(int x)
{
    cout << "Specialized emplate for int type: "
         << x << endl;
}

int main()
{
    example<char>('a');
    example<int>(100);
    example<float>(3.14);
}
```

Exemplu template specialization functions

```cpp
#include <iostream>
using namespace std;

template <class T>
class Example
{
public:
    Example()
   {
        cout << "Main template" << endl;
   }
};

template <>
class Example <int>
{
public:
    Example()
   {
        cout << "Specialized template object" << endl;
   }
};

int main()
{
    Example<int> a;
    Example<char> b;
    Example<float> c;
    return 0;
}
```

Exemplu template specialization classes

Template vs Overloading

Atât funcțiile supraîncărcate (overloaded) cât și template-urile sunt un bun exemplu de polimorfism în OOP.

Overloading e folosit când mai multe funcții realizează operații similare.

Template-urile sunt utilizate când mai multe funcții realizează operații identice.

În java se numesc generics

Summary

De Reținut

Template-urile sunt șabloane pentru funcții și clase care primesc ca parametrii tipuri de date.

Există template-uri în care putem realiza o logică diferită în funcție de tipul de date, iar conceptul se numește template specialization.

Template-urile sunt un exemplu de compile-time polimorfism.

Exerciții

Exerciții

1. **4.5p** Realizați un template pentru o funcție de concatenare care să trateze cazurile pentru char și int (exemplu: 1,2 -> 12 ; ‘a’,’b’ -> ‘ab’)

2. **4.5p** Realizați un template pentru o clasă de tip stivă (stack) care să aibă metodele de push și pop și să funcționeze atât pentru int-uri cât și pentru char-uri. Realizati acelasi lucru si pentru coada.

In ultima saptamana facem review la proiecte curs după laborator pentru fiecare echipă.